ReqLens Coach — Reflection

Graded Mini Project 14.3: Ship Your Own RAG · Live: https://reqlensai.com/coach · Repo: https://github.com/chamsedubai1/reqlens-ai

What I built. A public RAG chat at reqlensai.com/coach, inside ReqLens, my tool that scores agile user stories. Claude Haiku 5.5 answers through a `search_documents` tool call over hybrid retrieval (local BGE embeddings in pgvector + Postgres full-text), streamed with the Vercel AI SDK, with page-level sources under every answer. It runs on my Hostinger VPS next to the rest of ReqLens, not on Vercel.

Corpus and why. Three documents, 67 pages: the 2020 Scrum Guide (CC BY-SA) and two sets of my own study notes, one on Mike Cohn's *User Stories Applied*, one summarising two agile overviews (Kanban, Lean, XP, Hybrid). ReqLens tells a BA their story is weak; the Coach answers "how do I write it better", so the corpus is about story craft and delivery, and I know it well enough to spot a wrong answer. I first indexed the two overview PDFs themselves, then removed them because they are copyrighted, and wrote summaries instead. I chose not to index Cohn's book or PMBOK.

The trickiest decision was the model, and it came from measuring, not guessing. With Claude Opus 5.5 the answers were excellent (30/30 on my chapter eval), but after a tool call the whole answer arrived in one burst after ~7 seconds: I timed the stream and the text spread was under 10 ms. Sonnet 5.5 behaved the same; Haiku 5.5 and Opus 5 streamed properly. I picked Haiku 5.5: real streaming, first words in ~3 s, about $0.001 per answer on a public page, for one fewer correct citation. Chunking was the second: 1,200-character sentence-packed chunks with page ranges (700 scored the same), contents pages skipped, top-K raised from 6 to 8 when the eval showed correct passages ranked 7th to 9th.

What it does well: it knows where its knowledge ends. On a 55-question eval it scores 52/55 in two consecutive runs. It says "the documents don't cover that" for Jira set-up and PMBOK earned value instead of answering from general knowledge, attributes disagreements (2004-era 30-day sprints and the "three questions" vs the 2020 Scrum Guide), and corrects a false premise: asked why Cohn recommends numbering story cards, it quotes p. 9, "Do not number the cards." That only worked after I added keyword search; the embeddings alone never surfaced a line buried in a list of thirteen habits.

What it does badly: facts buried in long passages, and whole-document questions. "Who has the final say on priorities?" gets a sensible Scrum answer (the Product Owner) but misses the notes' line "the customer wins. Every time." A comparison table split across two chunks loses its heading, so the second half is rarely retrieved, and once the bot wrongly said the notes gave no figures. When my summary restates the Scrum Guide it sometimes cites the summary, not the Guide. Summaries of a whole book or counting mentions can only use retrieved passages; it says so.

With another week I would carry each section heading into its chunks (fixing the split table and giving citations like "Ch. 9, p. 11"), add a reranker for list-buried answers, add a document-outline tool for summaries and counts, replace my regex grader with a judged answer-quality eval, and add a usage page with questions, searches and latency.